

A Decentralized Voting System

Abstract.

This thesis presents the design and implementation of a decentralized voting system that runs on an Ethereum blockchain and targets small and medium-sized elections. The system aims to reduce reliance on a centralized server by making the election state and tally derivable from publicly verifiable on-chain data, while keeping individual choices hidden during the active voting period.

The proposed solution is a browser decentralized application with an Ethereum smart contract as the authoritative back-end. Each deployment of the Voting contract represents exactly one election instance and enforces a five-phase workflow: *Initialization*, *Registration*, *Commit*, *Reveal*, and *Finalized*. Eligibility is managed by a chairperson who registers voter addresses and configures proposals and deadlines during setup. Voters cast at most one ballot using a commit–reveal protocol: in the commit phase they submit a cryptographic commitment (hash) to their choice and a secret salt, and in the reveal phase they disclose the choice and salt so the contract can verify the commitment and count the vote. Deadlines bound the voting windows and support liveness by allowing progression after time expiry.

The system is implemented as a Solidity contract and a React web client that interacts through a standard wallet MetaMask. The smart contract is evaluated through systematic automated tests written in TypeScript and executed with Hardhat on a local development chain. The test suite covers normal execution paths and negative cases such as actions in the wrong phase, double voting attempts, invalid reveals, and expired deadlines, and checks global consistency invariants after state transitions. The results show that the system correctly enforces phase rules and eligibility constraints and produces a publicly verifiable tally under the assumed trust model. The source code repository is available at <https://github.com/Fantasy7780/voting-system>.

Keywords: blockchain; decentralized application; Ethereum; smart contract; React.

Zdecentralizowany system głosowania

Streszczenie.

Niniejsza praca przedstawia projekt oraz implementację zdecentralizowanego systemu głosowania działającego w łańcuchu bloków Ethereum, przeznaczonego dla wyborów o małej i średniej skali. Celem systemu jest ograniczenie zależności od scentralizowanego serwera poprzez umożliwienie wyprowadzenia stanu wyborów oraz wyniku z publicznie weryfikowalnych danych zapisanych w łańcuchu, przy jednoczesnym ukryciu indywidualnych wyborów w trakcie trwania głosowania.

Zaproponowane rozwiązanie ma postać przeglądarkowej aplikacji zdecentralizowanej z inteligentnym kontraktem Ethereum pełniącym rolę autorytatywnego zaplecza. Każde wdrożenie kontraktu Voting odpowiada dokładnie jednej instancji wyborów i wymusza pięciofazowy przebieg: *Initialization*, *Registration*, *Commit*, *Reveal* oraz *Finalized*. Uprawnienia są zarządzane przez przewodniczącego wyborów, który rejestruje adresy wyborców oraz konfiguruje propozycje i terminy podczas etapu przygotowania. Wyborcy oddają co najwyżej jeden głos z wykorzystaniem protokołu commit-reveal: w fazie commit przesyłają kryptograficzne zobowiązanie (hash) dotyczące wyboru oraz tajny salt, natomiast w fazie reveal ujawniają wybór i salt, aby kontrakt mógł zweryfikować zgodność zobowiązania i zliczyć głos. Terminy ograniczają okna głosowania i wspierają żywotność protokołu, umożliwiając przejście do kolejnych faz po upływie czasu.

System zaimplementowano jako kontrakt w języku Solidity oraz klienta webowego w React, komunikującego się za pośrednictwem standardowego portfela MetaMask. Inteligentny kontrakt oceniono przy użyciu systematycznych, zautomatyzowanych testów napisanych w TypeScript i uruchamianych z wykorzystaniem Hardhat na lokalnym łańcuchu deweloperskim. Zestaw testów obejmuje typowe ścieżki wykonania oraz przypadki negatywne, takie jak działania w niewłaściwej fazie, próby podwójnego głosowania, niepoprawne ujawnienia oraz przekroczone terminy, a także weryfikuje globalne niezmienniki spójności po przejściach stanów. Wyniki pokazują, że system poprawnie wymusza reguły faz oraz ograniczenia uprawnień i wytwarza publicznie weryfikowalny wynik w ramach przyjętego modelu zaufania. Repozytorium kodu źródłowego jest dostępne pod adresem <https://github.com/Fantasy7780/voting-system>.

Słowa kluczowe: blockchain; zdecentralizowana aplikacja; Ethereum; smart contract; React.

Contents

1. Introduction	7
1.1. Motivation	7
1.2. Problem and Approach	7
1.3. Contributions	8
2. Background	9
2.1. Bitcoin Basics	9
2.1.1. Block Structure	9
2.1.2. Proof-of-Work Consensus	10
2.1.3. Bitcoin Unspent Transaction Output Model	11
2.2. Ethereum	12
2.2.1. World Computer	12
2.2.2. Accounts	13
2.2.3. Ethereum Virtual Machine	13
2.3. Technology Stack	14
3. Design	17
3.1. System Architecture and Operation Overview	17
3.1.1. Actors	17
3.1.2. Runtime Components	18
3.1.3. Interaction Model	19
3.2. Smart Contract Lifecycle	19
3.2.1. Phases	19
3.2.2. Transition Rules and Triggers	21
3.2.3. Operational Walkthrough	21
3.3. Smart Contract Design	22
3.3.1. Contract Responsibilities	22
3.3.2. On-chain Data Model	23
3.3.3. Functions and API	23
3.3.4. Events and Off-chain Observability	25
3.4. Front-end Design	26
3.4.1. Front-end Responsibilities	26
3.4.2. Roles and UI Surface	26
3.4.3. Workflow Mapping	27
3.4.4. Client State Model	27
4. Implementation	29
4.1. Back-end implementation: on-chain Voting contract	29
4.1.1. Codebase and compilation target	29
4.1.2. Storage, access control, and phase gating	29

4.1.3. Lifecycle transitions	31
4.1.4. Administrative operations	32
4.1.5. Commit–reveal voting implementation	33
4.1.6. Read-only interfaces, error handling, and events	34
4.2. Front-end implementation: web client	35
4.2.1. Stack and configuration	36
4.2.2. Read/write data layer and receipt handling	36
4.2.3. Commit–reveal client support and local persistence	38
5. Testing	39
5.1. Testing overview	39
5.2. Tooling and environment	39
5.3. Test harness helpers	40
5.3.1. Constants and network bootstrap	40
5.3.2. Time control utilities	40
5.3.3. Deployment and canonical setup	41
5.3.4. Action wrappers with event assertions	41
5.3.5. Invariant checker	42
5.3.6. Minimal phase constructor	42
5.4. Phase–function matrix loop	42
5.5. Functions test logic	44
5.6. Global invariant checks	46
5.7. <code>changeState</code> failure coverage checklist	47
5.8. Test execution results	48
6. Conclusions	49
References	51
Source Code Repository	52
List of Figures	52
List of Tables	52

1. Introduction

This thesis presents the design and implementation of a decentralized voting system that runs on an Ethereum blockchain. The system targets small and medium-sized elections, such as internal votes in universities or companies, where participants expect a simple user experience but also want clear guarantees that the result has not been manipulated.

1.1. Motivation

In many organizations, elections are run with paper ballots or with simple web-based tools. In both cases, the vote is controlled by a central service. This service decides who is allowed to vote, records each ballot, and produces the final tally. As long as the service is honest and well maintained, this approach works in practice.

The problem is that all trust is concentrated in one place. A failure in the infrastructure, a software bug, or a malicious administrator can affect the result in ways that are hard to detect. Voters usually see only the final numbers, not the process that produced them, and they have little opportunity to verify that all ballots were handled correctly.

Public blockchains offer a different model. They maintain a shared ledger that is replicated by many nodes, and they execute smart contracts that define how state can change. Once data is written to the chain according to the contract rules, it is difficult to alter without leaving evidence on the ledger. This suggests that blockchains can be used as a transparent and tamper-resistant backend for voting.

At the same time, using a blockchain does not automatically solve all voting problems. All data on-chain is public, transactions have a cost, and the throughput and latency of the network are limited. A voting system that runs on a blockchain must therefore be designed to respect these constraints, and must protect ballot privacy even though the underlying ledger is public.

This thesis is motivated by the question of how far one can go with a relatively simple combination of smart contracts and a web front-end, without relying on advanced cryptography or specialized infrastructure, and still obtain better integrity and transparency guarantees than a basic centralized web application.

1.2. Problem and Approach

This thesis studies how to design and implement a practical voting system on an Ethereum blockchain that can serve as a backend for small and medium-sized elections.

More precisely, the work focuses on elections with a limited number of voters and options, under the following requirements:

- only registered voters are allowed to participate, and each voter can cast at most one ballot in a given election;

- individual choices should not be visible while the election is still in progress;
- the final tally should be derivable and verifiable from public on-chain data;
- the system should be simple enough to deploy and use with standard tools such as a web browser and a wallet.

The approach taken in this thesis is to meet these requirements with a commit–reveal protocol implemented as a set of smart contracts. In the commit phase, voters submit commitments to their choices in the form of cryptographic hashes. In the reveal phase, they publish the corresponding clear-text choices and secrets. The contracts check that each reveal matches a previous commitment and belongs to a registered voter, and then count the valid ballots.

In this design, the blockchain is the main source of truth for the state of each election: which voters are registered, which commitments have been made, which reveals have been accepted, and what the final tally is. A web-based front-end helps users interact with the contracts, but it does not store any authoritative voting data. The correctness of the election therefore depends primarily on the smart contracts and the underlying blockchain, rather than on a central application server.

1.3. Contributions

Within the scope described above, this thesis makes the following contributions:

- **System model.** It describes a concrete setting for small and medium-sized elections on an Ethereum-compatible blockchain, including the roles of organizers and voters, and the assumptions about the underlying network and adversary.
- **Protocol and contract design.** It specifies a voting protocol based on a commit–reveal pattern, and designs smart contracts that implement this protocol while keeping on-chain state compact.
- **Prototype implementation.** It provides a working decentralized application that combines the smart contracts with a web-based client, allowing voters to take part in elections using a standard wallet.
- **Evaluation through testing.** It evaluates the prototype using systematic tests that cover normal voting flows and edge cases such as double-voting attempts, missing reveals, and actions in the wrong phase of the election, and relates the observed behavior to the intended properties of the system.

These contributions are intended to be practical: the focus is on a complete but compact design that can be implemented and tested with standard tools.

2. Background

This section introduces the technical background needed for the rest of the thesis. It focuses on concepts that are directly used in the design and implementation of the voting system: the basic structure of a blockchain, the Bitcoin model of transactions and consensus, and, in later subsections, the Ethereum platform and key management.

2.1. Bitcoin Basics

Bitcoin is the first widely deployed public blockchain system. It was introduced as a peer-to-peer electronic cash system in which no central party controls the ledger [1]. Instead, many nodes maintain their own copies of the ledger and agree on a single chain of blocks through a consensus mechanism based on proof of work.

2.1.1. Block Structure

The Bitcoin ledger is organized into blocks. Each block has two main parts: a block header and a block body. The block body contains a list of transactions. The header contains a small set of fields that summarize the contents of the block and link it to the rest of the chain.

Figure 2.1 shows a simplified view of three consecutive blocks and the Merkle tree for one block. Each header stores, among other fields:

- a *version* field that identifies the block format;
- a *previous block hash*, which is the hash of the header of the previous block in the chain;
- a *Merkle root*, which summarizes all transactions in the block through a binary hash tree;
- a *timestamp* that records when the block was created;
- a *difficulty target* and a *nonce*, which are used by the proof-of-work mechanism described in the next subsection.

The previous block hash field creates an explicit link from each block to its predecessor. Because the hash is computed over the entire previous header, any change in an old block would also change this hash and break the link. In this way, the chain of blocks forms a tamper-evident history: modifying a past block would require recomputing the headers of all later blocks.

The Merkle root in the header plays a similar role for the transactions inside a block. Transactions are arranged in a tree, where each internal node stores the hash of its two children. The value in the root node, the Merkle root, depends on all transactions. If any transaction is added, removed, or modified, the Merkle root changes. This allows nodes to check quickly that the list of transactions they received for a block matches the value committed in the header, without having to recompute hashes for the entire chain.

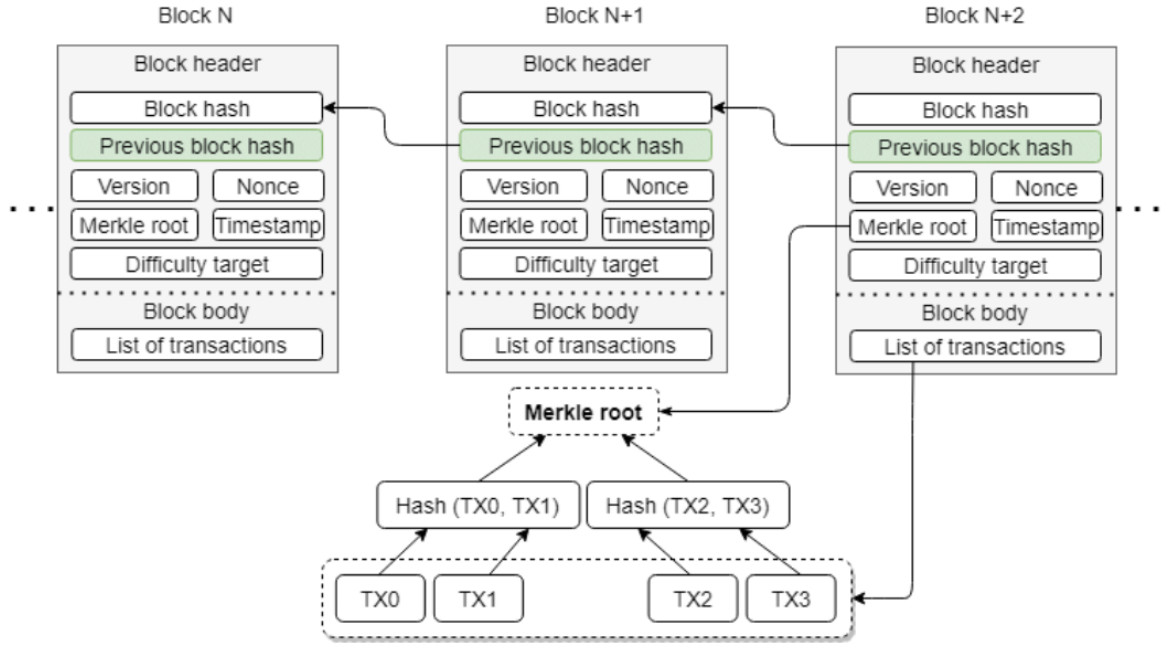


Figure 2.1. Bitcoin block structure[2].

2.1.2. Proof-of-Work Consensus

In Bitcoin, new blocks are created by nodes called miners. After constructing a candidate block (with a header as described in Section 2.1.1), a miner searches for a nonce value such that

$$\text{SHA256}(\text{SHA256}(\text{header})) < \text{target},$$

where *target* is a 256-bit unsigned integer that represents the current difficulty of the network [1]. A smaller target means that fewer hash values satisfy the inequality, so it is harder to find a valid nonce. In practice this is often described as requiring the hash to have more leading zero bits in its binary representation.

The proof-of-work process is essentially brute force. For each candidate nonce, the miner computes the hash of the header and checks whether it is below the target. Because SHA-256 behaves like a random function, each output is effectively a random 256-bit number. If the protocol parameters correspond to a difficulty level d , the expected amount of work for one valid block is proportional to 2^d hash evaluations.

In contrast, verification is cheap. Any node that receives a block only needs to evaluate the double hash once and check the inequality with the current target. The cost of verifying a solution is therefore constant-time with respect to the amount of work invested by the miner.

To keep the average time between blocks close to ten minutes, the Bitcoin protocol periodically adjusts the difficulty. Every 2016 blocks, the network measures how long it actually took to mine the last 2016 blocks and compares this with the expected time of two

weeks. If blocks were found too quickly, the difficulty is increased by lowering the target, and if blocks were found too slowly, the difficulty is decreased by raising the target. This feedback mechanism keeps the block interval relatively stable even as the total computing power of the network changes over time.

2.1.3. Bitcoin Unspent Transaction Output Model

Bitcoin represents value using the unspent transaction output (UTXO) model [1]. Instead of keeping an explicit balance for each address, the system keeps track of many small outputs, each of which can be spent at most once.

Each transaction has a set of *inputs* and a set of *outputs*. An output is a record that assigns a specific amount of bitcoin to a locking script, typically a condition that requires a valid digital signature from a given public key. As long as an output has not been used as an input to another transaction, it is an unspent transaction output, or UTXO. The global set of UTXOs at a given moment describes who can spend which amounts.

Figure 2.2 shows a simple example. In two previous blocks, there are outputs of 5 BTC(Bitcoin) and 4 BTC to the same address A. These two outputs are separate UTXOs. In the current block, a new transaction uses both of them as inputs, so the total input value is 9 BTC. The transaction creates two outputs: 6 BTC to address B and 3 BTC back to address A. After this transaction is confirmed, the original 5 BTC and 4 BTC outputs are marked as spent and are removed from the UTXO set, while the new 6 BTC and 3 BTC outputs are added as new UTXOs.

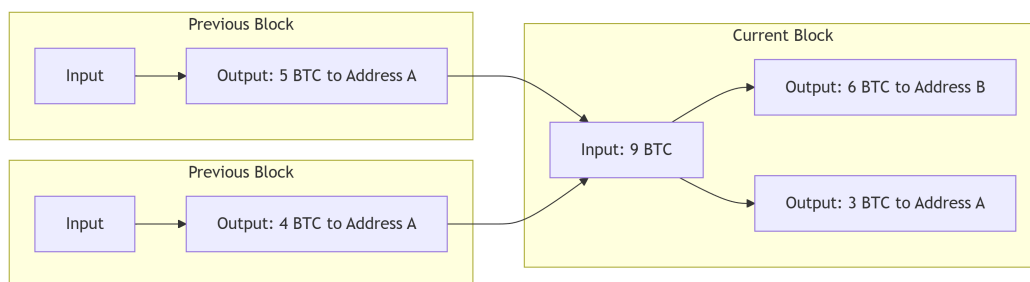


Figure 2.2. Example of UTXO.

2. Background

The wallet software that a user runs is responsible for selecting one or more UTXOs when the user wants to make a payment. Any small difference between the total value of the inputs and the total value of the outputs is treated as a transaction fee and is collected by the miner who includes the transaction in a block.

2.2. Ethereum

Ethereum is a general-purpose blockchain platform that extends the Bitcoin idea with programmable smart contracts. It was originally proposed by Buterin [3] and later specified in more detail in the Ethereum yellow paper by Wood [4]. It provides a shared network where users can deploy code and run decentralized applications without a central operator [5]. In contrast to Bitcoin's focus on payments, Ethereum is designed as a general execution platform.

2.2.1. World Computer

In Bitcoin, each block mainly records transfers of value: a sequence of transactions that move coins from one set of outputs to another. The blockchain acts as a distributed ledger, but the nodes do not run arbitrary application logic for each block.

Ethereum keeps the idea of a chain of blocks, but changes what a block means. Instead of only updating balances, a block describes how the state of a virtual computer changes when a list of transactions is applied [5]. All full nodes run the same programs on the same inputs and are expected to obtain the same resulting state.

Conceptually, Ethereum can be viewed as a transaction-based state machine. From one valid state, a sequence of transactions leads to a new valid state. Blocks group many transactions together. Each block header contains a hash that commits to the new global state after all transactions in the block have been applied. Because every full node executes the same transactions in the same order and checks the same hashes, all honest nodes agree on the same sequence of states.



Figure 2.3. A transaction causes a state change on Ethereum [5].

Ether (ETH) is the native cryptocurrency of Ethereum. It is used to pay for executing transactions on this world computer and to compensate the nodes that maintain the network.

2.2.2. Accounts

In Ethereum, an account is an entity with an ether (ETH) balance that can send transactions and receive funds [5]. All accounts live in the global state: conceptually, the Ethereum Virtual Machine (EVM) stores a large table that maps 20-byte addresses to account records and their balances.

There are two kinds of accounts:

- **Externally owned accounts (EOAs)**, which are controlled by private keys.
- **Contract accounts**, which are smart contracts deployed to the network and controlled by their code.

Both types of accounts can receive, hold, and send ether and tokens, and both can interact with deployed contracts. The essential difference is how actions are initiated.

An externally owned account is managed by a user. It is defined by a cryptographic key pair. The private key is a random 256-bit number, usually written as a 64-digit hexadecimal string. From this private key a public key is derived using the Elliptic Curve Digital Signature Algorithm (ECDSA). An address is then obtained by applying the Keccak-256 hash function to the encoded public key and taking the last 20 bytes of the result. The holder of the private key can sign transactions that spend funds or call contracts from the corresponding address, and nodes on the network verify these signatures with the public key.

A contract account is created when a smart contract is deployed in a transaction. It has an address and a balance, but no private key. Instead, it contains bytecode and persistent storage. A contract account never starts a transaction on its own. It reacts to incoming transactions: when a transaction targets the contract's address, the Ethereum Virtual Machine loads the stored bytecode and executes it, and the code may update storage, transfer ether or tokens, or call other contracts.

Internally, both EOAs and contract accounts are stored in the same way in the global state table: each entry keeps at least a nonce and a balance, and a contract account additionally stores its code and storage data [5]. The rest of the protocol specifies how transactions change these account records and how all nodes stay in agreement about their current values.

2.2.3. Ethereum Virtual Machine

The Ethereum Virtual Machine is the component that executes smart contract code [5]. It is a stack-based virtual machine with its own instruction set and memory model. All full nodes implement the EVM and use it to process contract-related transactions in the same way.

When a transaction calls a contract, the node:

- loads the contract's bytecode from the contract account;

2. Background

- creates an execution context with a program counter, a stack, memory, access to storage, and an initial amount of gas;
- executes the bytecode instruction by instruction, updating the context and deducting gas as operations are performed.

Gas is a unit that measures how many computation and storage resources a transaction consumes. The sender of a transaction specifies a gas limit and fee parameters. During execution, gas is spent according to the operations performed. If execution finishes successfully before the gas limit is reached, unused gas is refunded. If the gas limit is reached first, execution stops and the state changes from that call are reverted, but the gas consumed so far is still paid.

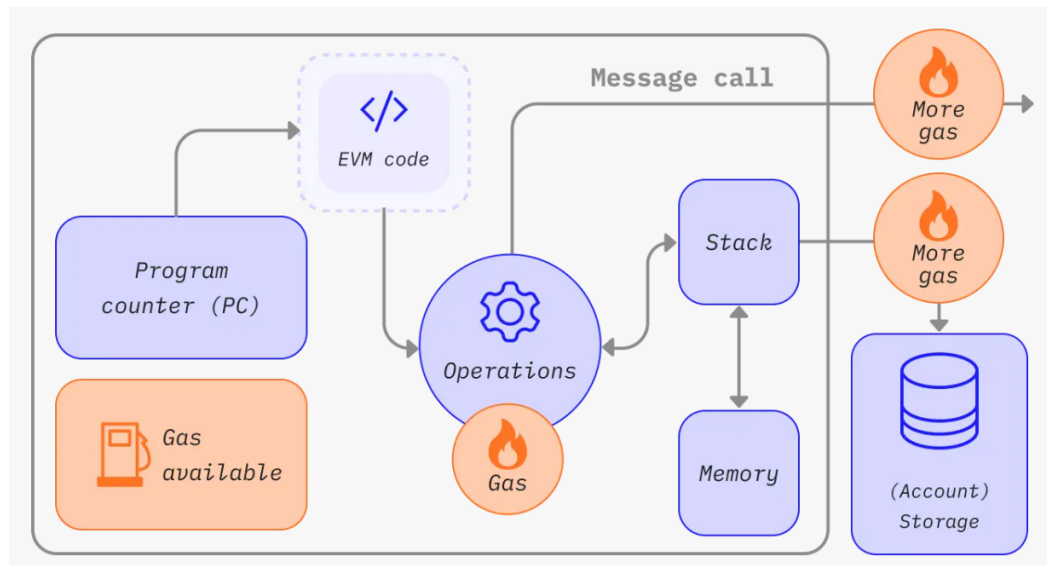


Figure 2.4. Ethereum Virtual Machine [5].

Smart contracts are programs stored in contract accounts and executed by the EVM. They are usually written in Solidity and compiled to bytecode before deployment. Once deployed, the code and public interface of a contract are visible to all participants, and the behavior of the contract is deterministic: given the same state and the same input, all nodes will compute the same result.

2.3. Technology Stack

The prototype voting system is implemented using a standard Ethereum technology stack. This subsection only lists the main tools and their roles. The later design and implementation chapters describe how they are used in detail.

- **Solidity** is used as the smart contract language. Ethereum supports several contract languages, as long as they compile to EVM bytecode, but Solidity is currently the most widely used and mature option, with extensive documentation and tooling.

All voting logic (election configuration, voter registration, commit and reveal, and tallying) is implemented in Solidity contracts. [6]

- **Hardhat** is used as the development framework. There are other frameworks for Ethereum development, but Hardhat offers an integrated local Ethereum node for development and testing, a flexible plugin system, and good support for TypeScript tooling. It compiles the contracts and provides scripts for deploying them. [7]
- **ethers.js** is used as the client library for interacting with the blockchain from the web application. Compared to older libraries such as `web3.js`, `ethers.js` provides a simpler, more consistent API and first-class TypeScript support. It offers abstractions for providers, signers, and contract calls. [8]
- **wagmi** and **viem** are used for blockchain interaction in the front-end. `wagmi` provides React hooks and wallet connector abstractions for reading from and writing to contracts, while `viem` provides a typed EVM client with ABI encoding/decoding for JSON-RPC calls. [9], [10]
- **React** is used for the web front-end. It is a widely adopted library for building applications and has a large options of components and tools. React makes it straightforward to structure the user interface. [11]
- **MetaMask** is used for key management and transaction signing. The front-end forwards prepared transactions to MetaMask, and the wallet signs and broadcasts them. [12]

3. Design

3.1. System Architecture and Operation Overview

The system is a browser-based decentralized application that uses an Ethereum smart contract as the authoritative back-end. Each deployment of the Voting contract represents exactly one election instance. The chairperson is the account that deploys the contract and is responsible for election setup and administration.

Figure 3.1 provides a high-level view of how the main actors and runtime components interact during normal operation. Solid arrows denote transactions that change on-chain state and dashed arrows denote read-only queries used to render the UI.

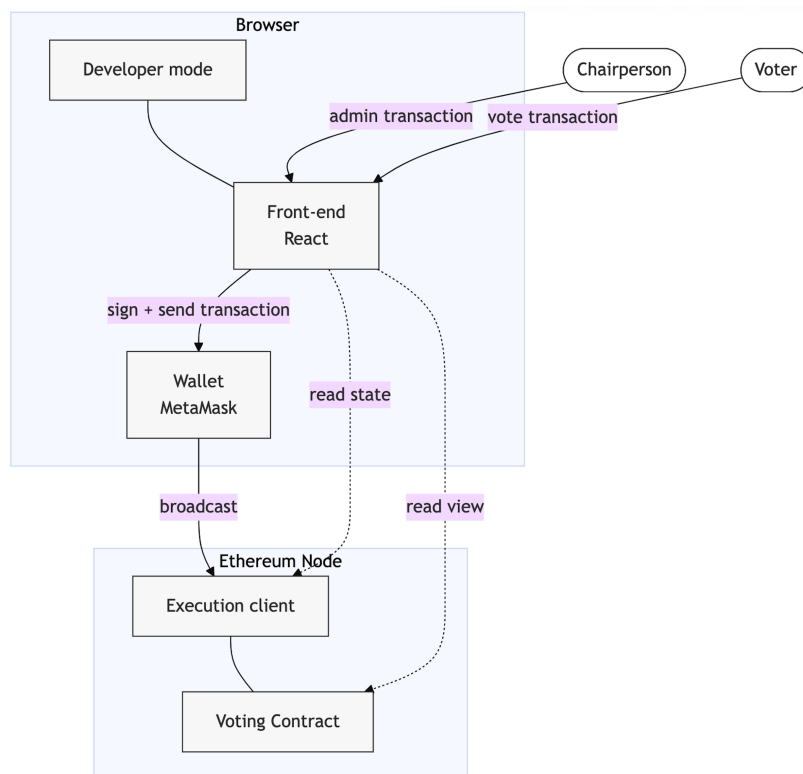


Figure 3.1. High-level architecture and interaction paths.

3.1.1. Actors

The system distinguishes two user roles.

- **Chairperson.** The organizer and administrator of an election instance (one election per contract). The chairperson deploys the contract, configures the election during the setup window, and controls progression of the workflow. In particular, the chairperson can register eligible voter addresses, create and edit proposals (e.g., add proposals and update proposal names) before voting starts, set the commit and reveal deadlines, monitor participation progress, and advance the contract to the next phase when the conditions are satisfied (or when deadlines have passed). When

an election is finished, the chairperson can finalize the workflow and publish the outcome by making the finalized on-chain state available for readout. To start a new election, the chairperson deploys a new contract instance.

- **Voter.** A registered participant who casts at most one ballot. The voter connects a wallet, checks registration status, and follows the commit–reveal protocol. Throughout the process, the voter can read the election state to confirm phase, deadlines, and final results.

However, to be user-friendly, only the important parts such as voting button can be seen by actors, unless they turn on Developer Mode.

3.1.2. Runtime Components

The architecture contains three main runtime components.

- **Front-end.** A React application running in the browser. It renders the current election state, guides users through the workflow, and prepares contract interactions. The UI also includes a developer mode that exposes additional diagnostics and advanced controls for testing and demonstration. Developer mode is a presentation layer feature: it changes what is shown to the user, but it does not alter the on-chain rules.
- **Wallet.** A standard wallet (MetaMask) manages the user's private keys and serves as the signing for all state-changing interactions. The front-end never reads or stores private keys. Instead, when the user triggers an operation that modifies on-chain state (e.g., commit, reveal, or chairperson administration), the front-end constructs the corresponding transaction payload and requests the wallet to sign it. The wallet displays the transaction details to the user, collects explicit confirmation, and then submits the signed transaction to the connected network. In this architecture, the wallet is the security boundary that binds actions to an Ethereum address.
- **Ethereum node.** During development, the system uses a local Ethereum node as a private blockchain environment. The node runs an execution client that validates and executes transactions, maintains the current chain state, and produces blocks. The Voting contract is deployed to this local chain, which provides a reproducible testbed with fast confirmation and controlled accounts. Both the front-end and the wallet connect to the same local chain in order to perform read-only queries (to render phase, deadlines, and results) and to broadcast signed transactions (to update contract state). Using a local chain isolates testing from public networks and avoids external dependencies while preserving the same transaction semantics as a real Ethereum deployment.

In a real deployment, the same interaction pattern applies on a public Ethereum network, but the execution environment changes. Transactions are propagated through the peer-to-peer network and included in blocks by validators, confirmation

latency becomes variable, and transaction ordering is no longer under local control. Read-only queries and transaction submission are typically performed through a remote node, while the contract state and event logs become globally observable to all nodes. Consequently, the contract logic and phase rules remain the same, but it can not be altered anymore. Anyone can write a front-end to use this contract, as long as they want to pay the gas. Reading is for free.

3.1.3. Interaction Model

The arrows in Figure 3.1 represent two interaction types.

- **Transactions (state-changing).** Solid arrows correspond to actions that modify contract state and therefore require a signed transaction and gas. Like voter commit and reveal operations, as well as chairperson administration. In the architecture, the front-end initiates the action, the wallet signs and submits it, and the node executes it against the contract.
- **Read-only queries.** Dashed arrows correspond to read-only calls used to display the current election status, proposals, and results. These queries do not modify chain state and do not require signatures.

3.2. Smart Contract Lifecycle

This section describes the lifecycle of a single election contract instance. The contract exposes an explicit phase variable and only permits operations that are valid for the current phase.

Figure 3.2 summarizes the contract lifecycle. Each deployment corresponds to exactly one election, and the contract progresses through five phases: *Initialization*, *Registration*, *Commit*, *Reveal*, and *Finalized*. Transitions are one-way and move forward by exactly one step.

3.2.1. Phases

Each phase defines a clear operational window and a corresponding set of permitted actions.

- **Initialization.** The contract exists but voting is not yet configured for participation. During this phase, the chairperson prepares the election parameters, primarily by creating the proposal set. The goal of this phase is to reach a consistent configuration before opening registration.
- **Registration.** The chairperson registers eligible voter addresses and finalizes the proposal set. This phase is also used to define time bounds for the subsequent voting steps by setting a commit deadline and a reveal deadline. Registration ends when the chairperson explicitly closes it by advancing the contract to the next phase.
- **Commit.** Registered voters submit a commitment as a transaction. A commitment represents an encoded vote choice and a secret salt, but it does not reveal the

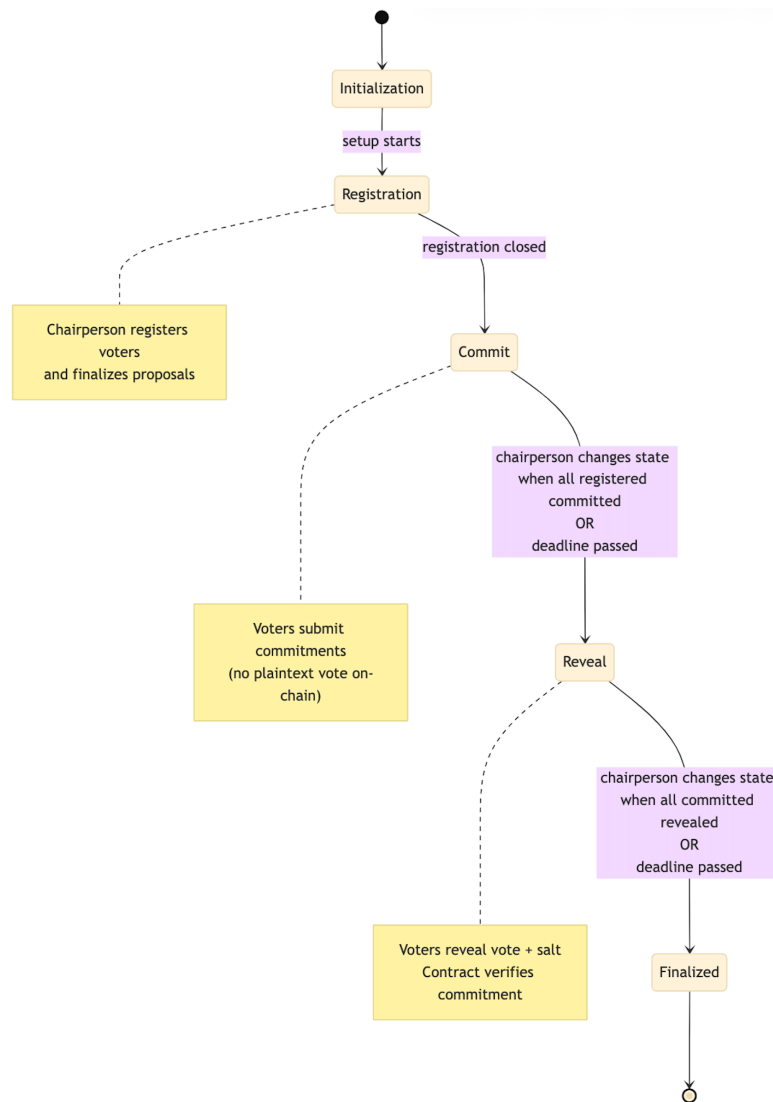


Figure 3.2. Phase state machine of a single election contract instance.

plaintext choice on-chain. The commit phase is bounded by a deadline: after the deadline, new commitments are rejected even if the phase variable has not yet advanced.

- **Reveal.** Voters reveal their plaintext choice together with the previously used salt. The contract verifies that the revealed pair matches the stored commitment before counting the vote. This phase is also bounded by a deadline: after it expires, new reveals are rejected.
- **Finalized.** The election is finished and the outcome becomes readable as finalized on-chain state. The contract exposes the final vote counts for each proposal in this phase, and no further voting-related state changes are accepted.

3.2.2. Transition Rules and Triggers

The contract enforces a strict progression policy: transitions are one-way and advance by exactly one phase. Operationally, there are two types of triggers.

- **Chairperson-driven advancement.** The chairperson advances the phase when conditions are satisfied. For early phases, this is the normal mechanism to open the next operational window (e.g., closing registration and starting commit).
- **Deadline-driven advancement.** For the two time-bounded voting phases, the lifecycle can advance once the corresponding deadline has passed. This supports voting system liveness: the election can progress even if not all participants complete their actions in time.

The voting phases can end in two ways, matching Figure 3.2. The contract can move from *Commit* to *Reveal* when either all registered voters have committed, or the commit deadline has passed. Similarly, it can move from *Reveal* to *Finalized* when either all committed voters have revealed, or the reveal deadline has passed. These conditions allow early progression when everyone has finished, while also preventing the voting system from stalling indefinitely.

3.2.3. Operational Walkthrough

The lifecycle can be executed as a simple operator procedure.

Chairperson procedure.

1. Deploy a new contract instance for the election. This defines a fresh election and assigns the deployer as chairperson.
2. In *Initialization*, configure the proposal set.
3. Advance to *Registration* to start setup for participation.
4. Register voter addresses and finalize proposals. Set the commit and reveal deadlines.
5. Close *Registration* by advancing to *Commit*.
6. Monitor participation. When either all registered voters have committed or the commit deadline has passed, advance to *Reveal* (or rely on deadline-driven advancement if applicable, in this case voters can advance to the next phase).
7. When either all committed voters have revealed or the reveal deadline has passed, advance to *Finalized* (or rely on deadline-driven advancement if applicable).
8. Publish the outcome by pointing users to the finalized on-chain results. A new election requires deploying a new contract instance.

Voter procedure.

1. After *Registration*, if voter's address is registered by the chairperson then the voter can see the proposals.

3. Design

2. During *Commit*, choose a proposal and vote before the deadline. The system will generate a fresh random salt, compute the commitment, and submit the commitment. The salt will be stored locally because it is required for the reveal step.
3. During *Reveal*, reveal the vote before the deadline. The system will submit the plaintext proposal index together with the original salt. The system also verifies the commitment match and counts the vote.
4. After *Finalized*, read the published tallies and winner(s) from contract state.

3.3. Smart Contract Design

The back-end of the system is a single Ethereum smart contract, `Voting`, which acts as the authoritative execution environment for the election. The contract defines the lifecycle of one election instance, stores the public election state, and enforces the commit–reveal workflow with strict phase control.

The contract is the only component that must be trusted for integrity: it validates who is allowed to perform administrative actions, when voters may commit or reveal, and how votes are counted. The front-end can be treated as a convenience layer for composing transactions and rendering state, but it does not define the election rules.

3.3.1. Contract Responsibilities

The contract is responsible for maintaining protocol correctness under the assumption that all on-chain state and event logs are publicly observable.

What the contract enforces. The contract enforces three categories of rules.

- **Workflow and timing.** The election progresses through an explicit phase variable (Phase and state) and time-bounded windows (`commitDeadline` and `revealDeadline`). Functions that change election state are guarded so they can only execute in the appropriate phase and within the corresponding time window.
- **Authorization.** Election administration is restricted to the chairperson (`chairperson`) via access control checks (e.g., `onlyChair`). This includes voter registration, proposal management, and phase progression.
- **Vote integrity under commit–reveal.** The contract stores a per-voter commitment (`commitHash`) and only counts a vote when the subsequent reveal is consistent with the stored commitment. In addition, the contract prevents multiple commits or multiple reveals from the same address by recording per-voter status flags.

What remains off-chain. Several aspects are intentionally out of scope for the smart contract and are handled off-chain.

First, key management and transaction signing are delegated to the user’s wallet. Second, the secret material required by commit–reveal (the salt) is not stored on-chain and must be preserved by the voter until the reveal phase. Third, user experience concerns

such as UI gating (e.g., developer mode), error messages, and state visualization are purely front-end responsibilities. As a result, any front-end can interact with the contract as long as it follows the public interface and pays transaction fees.

3.3.2. On-chain Data Model

The key storage elements are summarized in Table 3.1.

Table 3.1. Summary of on-chain storage elements and their roles.

Element	Purpose
enum Phase and state	Encodes the lifecycle stage of the election and gates valid operations (Initialization, Registration, Commit, Reveal, Finalized).
chairperson	Identifies the administrator address with permission to perform setup and governance actions.
Proposal [] proposals	Stores the proposal set as Proposal{name, voteCount}, enabling deterministic tallying and public readout of results.
mapping(address => Voter) voters	Stores per-address voting state as Voter{registered, committed, revealed, vote, commitHash}. This record binds each eligible address to at most one commit and at most one reveal.
address [] _voterAddrs	Maintains an iterable list of registered voter addresses for progress reporting and off-chain monitoring.
registeredCount, committedCount, revealedCount	Aggregated counters used to track protocol progress and support phase advancement conditions without scanning all voters.
commitDeadline, revealDeadline	Time bounds for the two voting steps. These variables define the latest allowed time for commit and reveal, and support liveness when participants do not complete actions.

In addition to persistent state, the contract emits events to make state changes observable for off-chain components. It will be explained later.

3.3.3. Functions and API

The Voting contract exposes a compact public API that is intended to be consumed by a thin client. The API can be grouped into administration, voting, liveness, and read-only queries.

Administration functions. The administration API defines how an election instance is configured and controlled by the chairperson. Functions such as register, addProposal, renameProposal, and setDeadlines are used to establish the voter registry, finalize the proposal set, and set the time bounds for voting. Lifecycle progression is explicitly controlled through changeState, which provides an operator-facing entry point for moving the contract to the next phase at the appropriate transition points.

Table 3.2. Public contract API grouped by function category.

Category	Function	Role	Phase
Administration	register addProposal renameProposal setDeadlines changeState	Chairperson	Initialization or Registration or transition points
Voting	commitVote revealVote	Registered voter	Commit (before commitDeadline); Reveal (before revealDeadline)
Liveness	advanceIfExpired	Any caller	Commit or Reveal, only after the corresponding deadline
Read-only queries	proposalNames voterAddresses getVoter computeCommitment results winners	Any reader	Always, except results/winners require Finalized

Voting functions. The voting API implements the commit–reveal protocol with a minimal surface: `commitVote` and `revealVote`. This separation is the core mechanism that delays disclosure of the vote until the reveal window.

Liveness function. To avoid protocol stalling when some participants do not complete their actions, the contract exposes `advanceIfExpired`. This function advances the phase after the relevant deadline has passed, making the progression rule observable and enforceable on-chain without requiring any privileged off-chain component. Operationally, it complements `changeState` by providing a deadline-based path to progress from *Commit* to *Reveal* and from *Reveal* to *Finalized*.

Read-only query functions. The read-only interface supports UI rendering and third-party verification without changing state. Functions such as `proposalNames`, `voterAddresses`, and `getVoter` provide public access to the proposal list and per-address status needed to display progress. The helper `computeCommitment` offers a contract-consistent reference for computing the commitment hash. Final outcome retrieval is provided via `results` and `winners`, which are only available once the contract reaches the *Finalized* phase, so the published outcome corresponds to a stable on-chain state.

3.3.4. Events and Off-chain Observability

In Ethereum, an event is recorded as an event log emitted during transaction execution. Event logs are indexed by the blockchain and can be efficiently queried by off-chain clients. Importantly, events are not part of the contract's persistent storage and are not readable by other contract code during execution. They are intended for off-chain observability, such as auditing.

While the current election state can always be reconstructed from storage via read-only calls, events provide two practical benefits. First, they enable reactive UI updates without continuous polling by notifying clients of state changes. Second, they provide a verifiable activity trail that supports monitoring, debugging, and auditing.

Table 3.3 summarizes the events emitted by the contract and their role in the system.

Table 3.3. Contract events and their off-chain purpose.

Event	Emitted when	Off-chain use
StateChanged	The phase is advanced via <code>changeState</code> .	Update the UI workflow state, switch visible actions, and record a phase history.
AutoAdvanced	A phase is advanced due to an expired deadline via <code>advanceIfExpired</code> .	Distinguish deadline-driven progression from manual advancement; useful for liveness monitoring.
VoterRegistered	The chairperson registers a voter via <code>register</code> .	Refresh voter lists and progress indicators; enable observers to verify the registry changes.
Committed	A voter submits a commitment via <code>commitVote</code> .	Update commit participation statistics and per-address status without scanning all voters.
Revealed	A voter reveals a vote via <code>revealVote</code> and it is accepted.	Update reveal participation statistics and enable real-time tally previews when appropriate.
ProposalAdded	A proposal is created via <code>addProposal</code> .	Refresh the proposal list presented to users and ensure consistent ordering across clients.
ProposalRenamed	A proposal name is updated via <code>renameProposal</code> .	Keep UI labels consistent with the canonical on-chain names and support auditability of edits.
DeadlinesSet	Deadlines are configured via <code>setDeadlines</code> .	Render countdowns and enforce client-side warnings about impending commit/reveal closure.

In addition to powering the default user interface, these events are also useful for developer mode: diagnostic views can display timelines and participation progress, while still treating storage reads as the source of truth.

3.4. Front-end Design

The front-end is implemented as a thin client running in the user's browser. It provides presentation and interaction logic while treating the smart contract as the sole authority for protocol rules and outcomes.

3.4.1. Front-end Responsibilities

Its responsibilities are:

- **State rendering.** Read the current contract state (phase, proposals, deadlines, participation counters, voter status, and final results) and present it in a human-readable way.
- **Workflow guidance.** Map the contract's phase machine into a clear user journey. The UI enables only actions that are expected to succeed in the current phase and provides warnings when windows are about to close, but does not act as a security control.
- **Transaction preparation.** Collect user inputs and format them into contract calls. All state transitions are executed as transactions signed by the user's wallet.
- **Client-side usability support.** Support commit–reveal participation by generating and persisting the secret salt needed for reveal, plus optional developer-mode diagnostics. These features improve usability but do not change protocol semantics.

3.4.2. Roles and UI Surface

The UI differentiates users by reading the on-chain chairperson address and comparing it with the connected wallet address. This avoids relying on client configuration for role assignment.

Voter (normal mode). Normal mode provides a minimal surface:

- The current phase, proposal list, deadlines, and the connected voter's status.
- Exactly one voting action per phase: *Commit* in the commit phase and *Reveal* in the reveal phase.

Commitment details (hashes and salt) are not exposed by default to reduce user error in the commit–reveal process.

Chairperson (normal mode). Normal mode exposes administrative controls required to run the election:

- Register voters (`register`) and set deadlines (`setDeadlines`) during registration.
- Progress the election (`changeState` or `advanceIfExpired`) when phase conditions are met.
- Monitor participation counters and deadlines throughout commit and reveal.

Developer mode. Developer mode is a presentation-layer feature intended for demonstrations and debugging:

- Show additional diagnostics (e.g., stored commitment).
- Allow advanced operations useful in experiments (e.g., switching the active contract address in the UI, deploying a fresh instance for a new round).

Developer mode does not bypass on-chain permissions. Any privileged action still reverts when invoked from a non-chair account.

3.4.3. Workflow Mapping

The UI uses the current on-chain phase as the primary driver for what is displayed and which controls are enabled.

- **Initialization.** Chairperson may add/rename proposals (`addProposal`, `renameProposal`). Voters have no voting actions.
- **Registration.** Chairperson registers eligible voters (`register`) and configures time windows (`setDeadlines`). The UI displays registration progress and configured deadlines.
- **Commit.** Registered voters submit a commitment via `commitVote`. The UI generates a random salt locally, computes the commitment using the contract's scheme (keccak256 over proposal index, salt, voter address, contract address, and chain identifier), and submits it as a transaction. Chairperson may advance when the commit window closes (`changeState/advanceIfExpired`).
- **Reveal.** Voters reveal their choice and salt via `revealVote`. The UI retrieves previously stored local metadata to reduce the risk of an incorrect reveal. Chairperson may finalize when the reveal window closes (`changeState/advanceIfExpired`).
- **Finalized.** The UI becomes read-only for voting and displays official outcomes using `results()` and `winners()`.

For consistency, the UI refreshes all relevant on-chain reads after a transaction receipt is confirmed. It avoids relying solely on events for correctness, while events remain useful for debugging and progress indication.

3.4.4. Client State Model

The front-end separates authoritative contract state from non-authoritative local state. Figure 3.3 illustrates the data flow and synchronization strategy.

The model consists of three layers:

- **On-chain snapshot cache.** A cached view of contract reads (phase, proposals, deadlines, counters, voter view, and final results when applicable). This snapshot drives all UI rendering.
- **Local UI state.** Transient interaction state such as selected proposal index, form inputs, transaction status, and developer-mode toggles.

3. Design

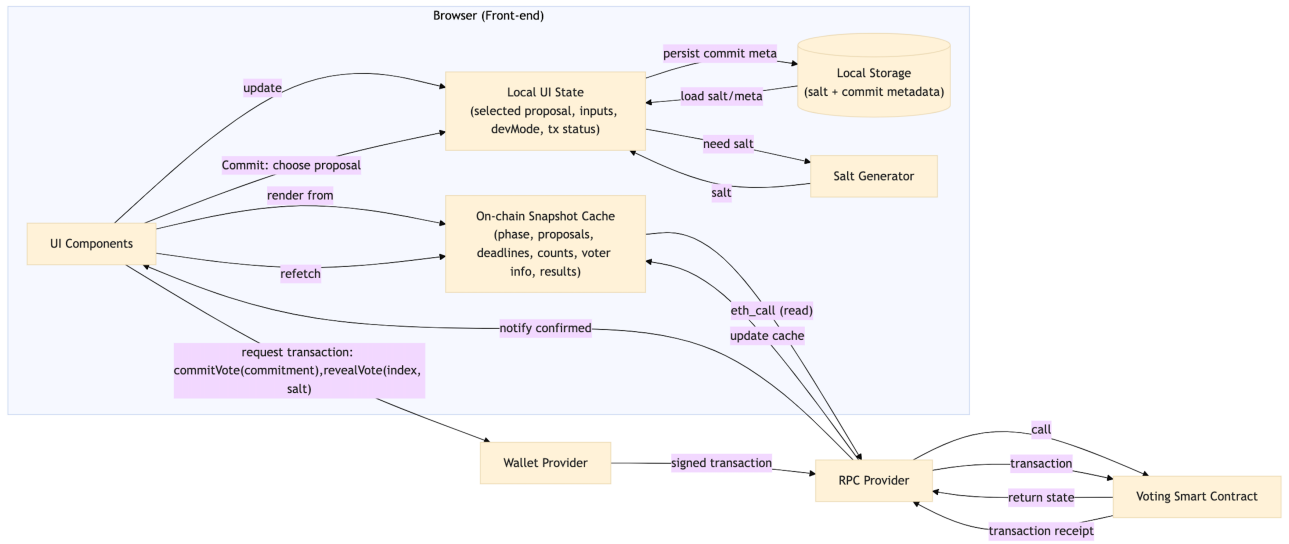


Figure 3.3. Front-end state model.

- **Local persistence.** Scoped local storage for commit–reveal metadata (salt and related fields) so that voters can reliably reveal later, including after reloads. This data is never treated as authoritative. It only supplies parameters for `revealVote`.

4. Implementation

This section complements the design discussion in Section 3.3 by focusing on *how* the system is implemented: code structure, storage layout, guard ordering, and the concrete logic inside each entry point. It covers both the on-chain `Voting` contract and the companion web client.

4.1. Back-end implementation: on-chain `Voting` contract

4.1.1. Codebase and compilation target

The on-chain logic is implemented as a single Solidity contract, `Voting`, intended to represent exactly one election instance per deployment.

The contract does not import third-party dependencies. All functionality (state machine control, registration, proposal administration, commit–reveal voting, and result readout) is implemented within `Voting`. This keeps the deployed artifact self-contained and reduces external assumptions during review.

Deployment-time configuration is handled by the constructor:

- The chairperson is recorded as `msg.sender` and stored in `chairperson`, which becomes the sole administrator for setup and phase transitions.
- The constructor takes `string[]` memory names and initializes proposals by pushing one `Proposal` struct per name with `voteCount` set to zero. This makes the initial proposal set a deployment parameter and avoids a mandatory post-deploy “seed proposals” transaction.
- The phase variable `state` starts at `Phase.Initialization`. This makes it explicit that only setup operations are allowed until the election is moved forward.

For observability, the implementation emits events for every state-changing operation that off-chain components may want to track. In addition, invalid calls fail with explicit revert strings, so tests and user interfaces can distinguish failure causes without ambiguity.

4.1.2. Storage, access control, and phase gating

The implementation organizes storage around three goals: constant-time progress checks, verifiable commit–reveal voting, and efficient state readout.

Core types. `enum Phase` encodes the lifecycle with five ordered values: `Initialization`, `Registration`, `Commit`, `Reveal`, and `Finalized`. The numeric ordering is used to enforce that transitions advance by exactly one step.

Each proposal is represented as: `struct Proposal { string name; uint256 voteCount; }`. All proposals are stored in a dynamic array `proposals`. The name is public metadata; the tally is updated only during successful reveals.

4. Implementation

Per-address state is stored as a struct that captures both eligibility and commit–reveal progress: `registered`, `committed`, `revealed`, `vote` (revealed index), and `commitHash` (submitted commitment).

Storage variables and enumerability. `address public chairperson` identifies the administrator, and `Phase public state` stores the current phase.

The authoritative per-voter record is stored in: `mapping(address => Voter) public voters`. Because mappings are not enumerable, the implementation also stores the list of registered addresses in `address[] private _voterAddrs` and exposes it via `voterAddresses()`. This enables off-chain iteration and supports invariant checks without requiring log reconstruction.

The contract maintains explicit counters: `registeredCount`, `committedCount`, and `revealedCount`. These provide constant-time predicates used by transition guards (e.g., “all registered voters have committed”) and by safety checks (e.g., monotonic progress across phases).

Two timestamps bound the voting windows: `commitDeadline` and `revealDeadline`. These are used both for runtime gating of `commitVote/revealVote` and for phase transition logic. Keeping deadlines in storage allows the contract to enforce time windows using `block.timestamp` without relying on external schedulers.

Read interfaces. Besides the compiler-generated getter for proposals, the implementation provides bulk readout helpers: `proposalNames()` for labels, and final-phase readouts `results()` and `winners()` for tally display. For voters, the contract exposes `getVoter(address)` to return a complete snapshot of the stored `Voter` fields in a stable signature.

Access control and phase gating. Correctness relies on rejecting calls that violate authorization, phase, or time constraints. The implementation centralizes these constraints using modifiers, which improves readability and makes failure modes stable and testable.

`onlyChair` restricts administrative operations to the chairperson by checking `msg.sender == chairperson`. Calls from other accounts revert with a specific reason string (e.g., “Only chairperson”). This modifier is applied to setup actions (registration, proposal management, deadline setting) and explicit phase transitions.

Phase-specific entry points use `inState(Phase p)` to check `state == p`. Calls in the wrong phase revert with “Wrong phase”. This guard is used for strictly phase-bound functions such as `register`, `commitVote`, `revealVote`, and final readout functions.

Commit and reveal windows are enforced with `commitWindowOpen` and `revealWindowOpen`. Each window modifier checks that the corresponding deadline is configured and that the current block time has not passed the deadline. The implementation distinguishes “not configured” from “expired” via different revert strings (e.g., “Commit deadline not set” vs. “Commit phase ended”).

Solidity evaluates modifiers in the order they appear in the function signature. As a result, the implementation's chosen ordering determines which revert reason is observed when multiple constraints are violated. The contract uses this feature to keep errors diagnosable (e.g., authorization failures are surfaced as authorization failures, phase failures as phase failures, and window failures as window failures). The test suite asserts these ordered failure modes.

Some setup-only behavior spans multiple phases (Initialization and Registration). For these cases, the implementation uses an explicit check inside the function body (e.g., allowing proposal changes only while state is one of the setup phases), and reverts with a dedicated message such as "Proposals are frozen" once voting begins.

4.1.3. Lifecycle transitions

Phase progression is implemented via two functions: an explicit chairperson-only transition (`changeState`) and an automatic time-triggered transition (`advanceIfExpired`). Both update state and emit events that allow off-chain listeners to track progress.

```
changeState(Phase newState)
```

It is restricted by `onlyChair` and enforces a strict step-by-step progression. The implementation compares enum integers and requires `newState` to equal `state + 1`; otherwise it reverts (e.g., "Must move by one"). This prevents backward transitions and phase skipping.

After the one-step rule, `changeState` applies target-specific guards:

- **Entering Commit.** The contract rejects entering the commit phase unless proposals exist, deadlines have been configured, deadline ordering is valid, and the commit deadline has not already passed at the time of entry. These checks prevent meaningless or already-expired commit phases.
- **Entering Reveal.** The contract rejects entering reveal while the commit step is still open. "Commit complete" is defined as either the commit deadline has expired, or all registered voters have committed early (checked using the counters). If neither holds, the transition reverts (e.g., "Commit still open").
- **Entering Finalized.** Similarly, entering finalized is rejected while reveal is still open. "Reveal complete" is defined as either the reveal deadline has expired, or all committed voters have revealed early. Otherwise, the transition reverts (e.g., "Reveal still open").

On success, the function updates state and emits `StateChanged` with the previous and next phase values. Logging both values enables off-chain reconstruction of the transition sequence directly from event logs.

4. Implementation

`advanceIfExpired()`

It provides a convenience path that advances the phase once the relevant deadline has passed. The implementation intentionally limits automatic transitions to the voting phases:

- In **Commit**, it requires `commitDeadline` to be set and already expired. If not expired, it reverts (e.g., "Not expired"). If expired, it sets `state = Reveal` and emits an automatic-advance event (e.g., `AutoAdvanced`) in addition to `StateChanged`.
- In **Reveal**, it requires `revealDeadline` to be set and expired. If expired, it sets `state = Finalized` and emits the same event pattern.
- In other phases, it rejects the call (e.g., "No auto-advance").

This function does not attempt to infer “all committed” or “all revealed” as a trigger; it is purely time-driven. Early completion is handled by the explicit `changeState` path (counter-based completion predicates), which avoids implicit transitions and keeps phase advancement explicit.

4.1.4. Administrative operations

Administrative operations implement the setup portion of the protocol: establishing eligible voters, defining proposals, and configuring deadlines. All of these are chairperson-only and are either limited to setup phases or become unavailable once voting begins.

`register(address voterAddr)`

It implements voter enrollment as a single write to `voters` plus append-only tracking for enumerability.

- **Preconditions.** The function is restricted to the Registration phase and the chairperson. It rejects the zero address and rejects attempts to re-register an already registered voter (preventing duplicate enrollment).
- **State updates.** On success, it sets `voters[voterAddr].registered = true`, appends the address to `_voterAddrs`, and increments `registeredCount`. Other `Voter` fields remain at their default values until `commit/reveal`.
- **Event emission.** It emits `VoterRegistered(voterAddr)` to support off-chain UI updates and auditability.

`addProposal(string name) and renameProposal(uint256 idx, string newName)`

Proposal administration is allowed only while the contract is in a setup phase (Initialization or Registration). After that, proposals are treated as frozen.

- **Preconditions.** Both functions are chairperson-only. They reject empty names. `renameProposal` additionally checks that the supplied index is within bounds (index bounds check).
- **Phase restriction.** Instead of a single-phase modifier, the implementation explicitly checks that state is a setup phase. If not, it reverts with a dedicated error (e.g., "Proposals are frozen").
- **State updates and events.** `addProposal` pushes a new `Proposal(name, 0)` and emits `ProposalAdded(index, name)`. `renameProposal` writes the new name into `proposals[index].name` and emits `ProposalRenamed(index, oldName, newName)`.

```
setDeadlines(uint256 commitUntil, uint256 revealUntil)
```

It configures the time bounds for the election.

- **Preconditions.** The function is chairperson-only and restricted to the Registration phase. It rejects deadlines that are in the past relative to `block.timestamp`, and rejects invalid ordering (e.g., `commitUntil >= revealUntil`).
- **State updates.** On success, it writes `commitDeadline` and `revealDeadline` into storage. These values are then used by both the voting window modifiers and the transition logic.
- **Event emission.** It emits `DeadlinesSet(commitUntil, revealUntil)`.

4.1.5. Commit–reveal voting implementation

As described in Section 3.3, the voting protocol is implemented as a standard commit–reveal scheme to mitigate front-running of vote choices. The commit phase stores a hash commitment; the reveal phase discloses the vote and a salt, which must reproduce the stored commitment.

```
computeCommitment(uint256 proposalIndex, bytes32 salt)
```

The contract provides a helper for generating the commitment hash using on-chain encoding rules. The implementation computes: `keccak256(abi.encode(proposalIndex, salt, msg.sender, address(this), block.chainid))`.

Including `msg.sender` binds a commitment to a specific voter address (binding property). Including `address(this)` and `block.chainid` provides domain separation: the same vote and salt cannot be replayed across contracts or across chains while producing the same commitment.

```
commitVote(bytes32 commitment)
```

It records a voter's commitment during the Commit phase.

- **Preconditions.** The function requires `state == Commit` and an open commit window (deadline set and not expired). It rejects callers that are not registered.

It rejects the zero hash commitment and rejects duplicate commits (a voter may commit at most once).

- **State updates.** On success, it writes the commitment into `voters[msg.sender].commitHash`, sets `committed = true`, and increments `committedCount`. It does not modify proposal tallies.
- **Event emission.** It emits `Committed(voter, commitment)`.

The contract does not attempt to validate the semantic content of the commitment at commit time; correctness is enforced at reveal time by recomputation and comparison.

`revealVote(uint256 proposalIndex, bytes32 salt)`

It opens a commitment and counts the vote during the Reveal phase.

- **Preconditions.** The function requires `state == Reveal` and an open reveal window. It rejects callers that are not registered. It rejects reveals without a stored commitment (no prior commit), rejects out-of-range proposal indices (index bounds check), rejects commitment mismatches, and rejects duplicate reveals.
- **Commitment verification.** The function recomputes the expected hash using the same tuple as `computeCommitment` (index, salt, voter, contract address, chain id) and compares it against the stored `commitHash`. If the values differ, it reverts (e.g., "Commit mismatch"). This is the core soundness check of the scheme.
- **State updates and tally.** On success, it records the revealed choice by setting `voters[msg.sender].vote = proposalIndex` and `revealed = true`, increments `proposals[proposalIndex].voteCount`, and increments `revealedCount`. This ensures that the global revealed counter matches the sum of per-proposal counts, which is also enforced by the invariant checks in Testing.
- **Event emission.** It emits `Revealed(voter, proposalIndex)`.

4.1.6. Read-only interfaces, error handling, and events

The implementation includes read-only helpers (view functions) to support UI rendering and auditing without requiring extensive event parsing.

Voter and proposal readout. `proposalNames()` builds and returns a `string[]` containing all proposal names in array order. Returning names in bulk avoids repeated per-index calls when rendering the ballot.

`voterAddresses()` returns the registered address list stored in `_voterAddrs`. `getVoter(address)` returns the stored voter record fields as a tuple. These functions are used for off-chain progress displays (e.g., how many voters have committed/revealed) and for audit/debug tooling.

Finalized results: results and winners. Final tally readout is restricted to the Finalized phase to avoid exposing partial results mid-election.

`results()` returns two arrays of equal length: proposal names and their corresponding vote counts, aligned by index. Internally it allocates memory arrays sized to `proposals.length` and fills them in a single loop.

`winners()` returns a list of proposal indices that achieve the maximum vote count, supporting ties (tie handling). The implementation uses a two-pass scan: the first pass computes the maximum tally, and the second pass collects all indices whose `voteCount` equals that maximum. This is linear in the number of proposals and avoids sorting.

Explicit revert reasons. The contract is designed to make incorrect usage diagnosable and off-chain state tracking straightforward.

The implementation uses `require` with explicit revert strings for distinct failure cases. This supports: precise negative testing (asserting the expected revert reason), clear UI error reporting, and easier auditing of guard coverage.

Revert reasons are grouped by purpose:

- **Authorization and phase.** Unauthorized callers revert with an authorization message, and wrong-phase calls revert with a phase message.
- **Setup validation.** Proposal and deadline configuration failures have dedicated messages (e.g., empty names, invalid indices, invalid deadline ordering, deadlines in the past, proposals frozen).
- **Commit/reveal validation.** The voting functions distinguish missing registration, missing commitments, bad commitments (zero hash), window expiration, commitment mismatches, and repeated commit/reveal attempts.

Events for off-chain state tracking. The implementation emits events for all state-changing actions relevant to observers:

- **Phase changes.** `StateChanged` is emitted for explicit and automatic transitions; `AutoAdvanced` distinguishes the time-driven path.
- **Setup actions.** `VoterRegistered`, `DeadlinesSet`, `ProposalAdded`, and `ProposalRenamed` allow off-chain systems to maintain an index of the election configuration.
- **Voting actions.** `Committed` and `Revealed` provide an auditable trace of participation without exposing vote choices at commit time.

The event parameter choices are aligned with off-chain reconstruction needs: addresses and indices are logged so that a listener can map actions back to stored state (e.g., which voter revealed which proposal index).

4.2. Front-end implementation: web client

This subsection describes how the browser client is implemented to realize the front-end design in Section 3.4. The implementation is contained in the `web/` directory as a Next.js application and interacts with the deployed `Voting` contract via wallet-signed transactions.

4.2.1. Stack and configuration

The client is built with Next.js (App Router), React, and TypeScript. Contract interaction is implemented using wagmi and viem, and read caching is provided by TanStack Query.

The runtime is initialized at the application root:

- `app/layout.tsx` defines the HTML shell and injects global CSS.
- `app/providers.tsx` wraps the application with `WagmiProvider` and `QueryClientProvider`.
The query client is created once per session to avoid cache resets on re-render.
- `src/lib/wagmi.ts` defines the wagmi configuration: an injected wallet connector (e.g., MetaMask) and an HTTP transport to the configured RPC.

Because wallet state is only available in the browser, `VotingPage` delays rendering until the component is mounted. This avoids hydration mismatches in server-rendered routes and ensures that wallet connection information is read consistently in the client environment.

Contract binding and configuration. The front-end binds to a specific contract instance through environment configuration and an optional UI override:

- `NEXT_PUBLIC_CONTRACT_ADDRESS` provides the default contract address at startup.
- `NEXT_PUBLIC_CHAIN_ID` defines the expected chain identifier used both for wallet/network checks and for commitment computation.
- `NEXT_PUBLIC_RPC_URL` (read by the wagmi configuration) selects the RPC endpoint when the configured chain id matches.

Contract ABI and bytecode are loaded from `src/abi/Voting.json` (`src/features/voting/contract`). enabling:

- typed read/write calls against deployed instances; and
- deployment of a fresh contract instance for a new round in developer mode.

The active contract address is stored in the front-end context (`VotingContext`) and can be changed at runtime through the `ContractSection` UI. Address validation is performed client-side using `isAddress`, but the authoritative correctness of interactions remains anchored in successful contract execution and receipt confirmation.

4.2.2. Read/write data layer and receipt handling

On-chain state is read through the `useVotingSnapshot` hook (`src/features/voting/hooks/useVotingSnapshot`). The hook issues a set of `useReadContract` calls and aggregates their results into a single snapshot object consumed by UI components.

The snapshot includes:

- the current phase (`state`);
- proposal labels (`proposalNames`);
- chairperson address (`chairperson`);
- participation counters (`registeredCount`, `committedCount`, `revealedCount`);

- deadlines (`commitDeadline`, `revealDeadline`);
- per-account voter view (`voters(account)`) when a wallet is connected; and
- final readouts (`results`, `winners`) only when the phase is `Finalized`.

Reads are conditionally enabled. In particular, final tallies are fetched only when the cached phase equals `Finalized`, which avoids unnecessary calls in earlier phases and matches the read interfaces exposed by the contract.

For consistency, `useVotingSnapshot` exposes a `refetchAll` function that triggers all reads in parallel. The front-end uses receipt-driven refresh: once a state-changing transaction is confirmed, the UI refetches the snapshot to converge to the on-chain truth rather than attempting to predict state locally.

As a compatibility measure, the snapshot layer also performs feature detection for proposal administration by scanning the ABI for `addProposal` and `renameProposal`. This allows the UI to hide management controls when the connected contract instance does not expose these entry points.

Transaction write layer and receipt handling. State-changing operations are implemented in `useVotingActions` (`src/features/voting/hooks/useVotingActions.ts`). This hook provides a uniform execution wrapper that:

- clears prior errors;
- executes a write call via `writeContractAsync`; and
- captures failures using a normalization function (error normalization) that prefers concise wallet/library messages (e.g., `shortMessage`).

Transaction progress is tracked using:

- `txHash` and `isPending` from `useWriteContract`; and
- `isMining` and `txOK` from `useWaitForTransactionReceipt`.

The UI surfaces these states through a centralized banner component and by disabling relevant controls while a transaction is pending or mining. In `VotingContext`, a successful receipt (`txOK`) triggers `snapshot.refetchAll()`, ensuring that the rendered state matches the contract after each confirmed transaction.

The action layer exposes:

- voter actions: `commitVote` and `revealVote`;
- chairperson actions: `register`, `setDeadlines`, `changeState`, and `advanceIfExpired`;
- optional proposal management: `addProposal` and `renameProposal`; and
- developer-mode deployment: `deployNewRound`, which deploys a new `Voting` instance via `walletClient.deployContract` and returns the created contract address after receipt confirmation.

All writes are ultimately authorized by the wallet signature boundary, and all security checks are enforced by contract reverts. Client-side prechecks (address format, numeric

ranges, missing inputs) are used to improve usability and reduce obviously invalid submissions, but are not relied upon for enforcement.

4.2.3. Commit–reveal client support and local persistence

Commit–reveal voting requires a per-voter secret salt to be preserved from commit to reveal. The front-end implements this using:

- a cryptographically secure pseudorandom number generator (CSPRNG) via `crypto.getRandomValues` to create a 32-byte salt (`randSalt32` in `utils.ts`);
- a deterministic commitment computation that reproduces the contract's `computeCommitment` encoding and `keccak256` hash using `viem`'s ABI encoding utilities and `keccak256` (the tuple includes the same fields: `index`, `salt`, `voter`, `contract`, `chainId`).

Salt and commit metadata are persisted in `localStorage` using scoped, namespaced keys (`storage.ts`):

- `salt:<chainId>:<contract>:<account>:<electionId>` stores the raw salt; and
- `commitmeta:<chainId>:<contract>:<account>` stores a structured commit record (`index`, `salt`, `commitment`).

On commit, the action layer validates the salt format (`bytes32` validator), computes the commitment, stores the metadata locally, and submits `commitVote(commitment)`. On reveal, the front-end attempts to restore metadata automatically:

- If commit metadata exists, it overrides the UI selection to use the recorded proposal index and salt, and it checks that the stored contract and `chainId` match the current context. Mismatches are reported with explicit client errors to prevent accidental reveals on the wrong contract instance or network.
- If metadata does not exist, the client falls back to the stored salt (if present) or manual input (typically exposed in developer mode).

To support device changes and recovery after clearing browser storage, the client implements optional backup and restore:

- `collectCurrent` exports the current account-scoped items for the active contract; `collectAll` exports all stored salts and commit metadata across scopes.
- `importBackup` validates and restores items from a JSON blob.
- `downloadText` provides a browser download helper for exporting backup JSON.

These persistence features are explicitly non-authoritative: they only supply inputs required to call `revealVote`. Correctness of the vote remains dependent on the contract's commitment verification and phase/window checks.

5. Testing

5.1. Testing overview

The contract is tested with an automated, code-driven approach that mirrors how the system is expected to be used on-chain: construct a minimal valid baseline state for a given election phase, exercise one contract entry point under different roles and inputs, and re-check global consistency properties after each interaction. This structure provides function-level confidence (revert reasons, event emission, and phase gating) while also guarding against cross-function state corruption.

The test scope is deliberately **limited to the smart contract**. End-to-end front-end testing is comparatively difficult to make deterministic in this project because it depends on external wallet interaction, asynchronous UI behavior, and user-driven flows that are outside the EVM execution model. Moreover, once deployed, a smart contract's bytecode is effectively immutable, so correctness must be established before deployment. For these reasons, the evaluation focuses on systematic on-chain tests of the contract interface.

The main test suite, `Voting - matrix + invariants`, iterates over all phases and runs a phase-function matrix: for each phase, it builds the required baseline using `buildPhase`, executes one test per public entry point, and finally invokes the global invariant checks. This yields uniform coverage across phases without duplicating setup code and makes phase-dependent behavior explicit.

A second suite focuses specifically on `changeState` failure modes. It tracks which expected revert reasons have been observed and asserts that all transition guards are covered by at least one concrete scenario. This prevents accidental weakening of the state machine by ensuring that each guard remains reachable and test-covered as the implementation evolves.

5.2. Tooling and environment

All tests are written in TypeScript and executed with Hardhat's test runner on a local simulated chain (Hardhat Network). Hardhat provides the runtime environment (HRE) used by the test code to connect to the in-process network, deploy contracts, issue transactions from multiple accounts, assert emitted events, and validate revert reasons with Ethereum-specific Chai matchers. Hardhat's testing workflow and recommended setup (TypeScript + ethers.js + Mocha/Chai) are documented in the official Hardhat documentation [13].

The test file imports `hre` and obtains an ethers instance by calling `hre.network.connect()`, storing the result in a shared variable (`ethersX`). This allows all helper routines to use a single provider and signer set for deployment, timestamp inspection, and JSON-RPC(Remote Procedure Call) calls. To test deadline-dependent logic, the

environment relies on Hardhat Network's RPC methods for time control: `evm_setNextBlockTimestamp` is used to schedule the next block timestamp and `evm_mine` is used to mine that block immediately. This makes commit/reveal window tests deterministic.

5.3. Test harness helpers

This subsection explains the helper functions defined at the top of the test file.

5.3.1. Constants and network bootstrap

P (phase constants)

The tests define a constant map `P` that assigns explicit numeric values to each phase: `Initialization = 0`, `Registration = 1`, `Commit = 2`, `Reveal = 3`, and `Finalized = 4`. The comment in the test file notes that this ordering must match the Solidity enum order. This is necessary because `changeState` in the contract enforces a strict “forward by one” transition by comparing enum integers (`uint8(newState) == uint8(state) + 1`).

SALT_OK and SALT_BAD

Two fixed 32-byte salts are used to make commitment hashes deterministic and to intentionally trigger a commit mismatch in negative tests. The “bad salt” case is particularly important because `revealVote` recomputes the hash on-chain and compares it to the stored commitment.

before(...) network connection and ethersX

The `before` hook connects to the active Hardhat network via `hre.network.connect()` and stores the returned ethers API in `ethersX`. All helper routines below use this shared handle to obtain signers, deploy contracts, read blocks, and send RPC calls.

5.3.2. Time control utilities

rpcSend(method, params)

A thin wrapper around `ethersX.provider.send` that is used to call Hardhat Network JSON-RPC methods directly. Keeping it as a wrapper improves readability at call sites and centralizes the provider cast.

nowTs()

It reads the latest block via `provider.getBlock("latest")` and returns its timestamp as a JavaScript number. This helper is used as the baseline when computing deadline values.

mineAt(ts)

It sets the timestamp for the next block (`evm_setNextBlockTimestamp`) and immediately

mines it (`evm_mine`). Tests that need to check commit or reveal window boundaries use `mineAt(deadline + 1)` to force the contract into an “expired” state deterministically.

5.3.3. Deployment and canonical setup

`deployFresh(names)`

It deploys a new Voting contract instance and returns both the deployed contract handle and a fixed set of signers (`chair`, `a1`, `a2`, `a3`, `a4`). The default proposal list is `["Alice", "Bob"]`, but many tests override this to exercise edge cases such as “no proposals”. The helper waits for deployment completion (`waitForDeployment`) to ensure subsequent calls are executed against a deployed address.

5.3.4. Action wrappers with event assertions

`step(voting, chair, to)`

It transits the election by exactly one phase using the chairperson: `changeState(to)`. It asserts that `StateChanged(from, to)` is emitted and that `voting.state()` equals to afterwards. By encoding these checks once, all tests that need to advance phases can be written in a uniform and auditable style.

`registerVoter(voting, chair, voterAddr)`

It calls `register(voterAddr)` and asserts that the contract emits `VoterRegistered(voterAddr)`. This ensures that tests do not only check storage counters indirectly, but also confirm the observable event interface.

`setValidDeadlines(voting, chair, commitDeltaSec, revealDeltaSec)`

It computes `commitUntil` and `revealUntil` relative to `nowTs()`, calls `setDeadlines`, and asserts `DeadlinesSet(commitUntil, revealUntil)`. The helper returns the computed deadlines, which allows other tests to mine blocks around those timestamps without re-deriving them.

`doCommit(voting, voter, proposalIndex, salt)`

It implements the commit operation in two steps: it first calls the contract’s `computeCommitment` and then submits that hash to `commitVote`. Importantly, the helper does *not* reimplement the hashing off-chain; it uses the contract’s own helper to avoid mismatches in encoding rules. It also asserts that the `Committed(voter, commitment)` event is emitted and returns the computed commitment.

`doReveal(voting, voter, proposalIndex, salt)`

It calls `revealVote` with the revealed index and salt and asserts that `Revealed(voter, proposalIndex)` is emitted. This wrapper is used in both success-path flows and in the phase builder described below.

5.3.5. Invariant checker

```
assertInvariants(voting)
```

This function ensure that tests continuously enforce consistency. More details will be discussed later.

5.3.6. Minimal phase constructor

```
buildPhase(target)
```

The matrix-based suite needs a repeatable way to obtain a contract in each phase with just enough valid setup to make subsequent calls meaningful. The `buildPhase` helper does exactly that. It deploys a fresh contract with two proposals and chooses two identities: `rv` (a voter that will be registered when the phase requires it) and `uv` (an unregistered voter). It then conditionally performs the minimal sequence of steps required to reach `target`:

- For Registration: advance once from Initialization.
- For Commit: enter registration, register `rv`, set valid deadlines, then advance to commit.
- For Reveal: additionally commit once (with `SALT_OK`) and advance.
- For Finalized: additionally reveal once and advance.

By centralizing phase construction, the functions tests can focus on behavior (phase gating, role checks, revert reasons, event emission, and deadline boundary cases) rather than on repetitive setup code.

5.4. Phase-function matrix loop

The core of the test code is a nested phase loop that turns the contract workflow into a systematic test matrix. The suite defines an ordered list of phase keys and iterates over them:

```
const PHASES: PhaseKey[] = [  
  "Initialization", "Registration", "Commit", "Reveal", "Finalized"  
];
```

```
for (const phase of PHASES) {  
  describe(`phase = ${phase}`, function () {  
    it("register(): ...", async () => { ... });  
    it("setDeadlines(): ...", async () => { ... });  
    it("addProposal(): ...", async () => { ... });  
    it("renameProposal(): ...", async () => { ... });  
    it("commitVote(): ...", async () => { ... });  
    it("revealVote(): ...", async () => { ... });  
    it("advanceIfExpired(): ...", async () => { ... });  
  });  
}
```

```

    it("results() / winners(): ...", async () => { ... });
  });
}

```

Each `it(...)` block is written as a *single* test case that can run under *any* phase. The first statement inside each test is a call to `buildPhase(phase)` (Section 5.3), which deploys a fresh contract and performs the minimum sequence of setup transactions needed to reach the requested phase. The helper returns three relevant identities: the chairperson, `rv` (a registered voter in phases where registration has already happened), and `uv` (an unregistered voter). This allows the same test body to exercise both role-dependent and registration-dependent checks without redefining accounts in every case.

The matrix tests follow an explicit early-return pattern to keep the phase logic auditable. For example, each test begins by checking whether the current phase is one of the allowed phases for the function. If not, it asserts the expected revert behavior and returns immediately:

```

const { voting, chair, rv, uv } = await buildPhase(phase);

if (phase !== "Registration") {
  await expect(voting.connect(chair).register(uv.address))
    .to.be.revertedWith("Wrong phase");
  await expect(voting.connect(rv).register(uv.address))
    .to.be.revertedWith("Only chairperson");
  await assertInvariants(voting);
  return;
}

```

This structure makes two properties visible in the test code.

First, it captures *phase gating* as a concrete matrix: each function is attempted in every phase, so “wrong phase” behavior cannot be missed by accidental under-testing.

Second, it captures *check ordering*. For chairperson-only functions such as `register` and `setDeadlines`, the contract checks `onlyChair` before `inState`. Therefore, in a non-Registration phase, the chairperson call reaches the phase check and reverts with “Wrong phase”, while a non-chair call reverts earlier with “Only chairperson”. The test asserts both outcomes to confirm that the guards are not only present but ordered as intended.

After every phase-specific branch (success path or revert path), the test calls `assertInvariants(voting)`. This makes the matrix suite a regression guard for both successful state updates and reverting transactions: any unexpected storage change that breaks global consistency is caught even if the individual assertion for the function appears correct.

Two additional cases complement the phase loop. The suite contains a "sanity: deployment getters" test that validates the initial deployment state and basic getters, and a longer end-to-end flow test that runs through an entire election instance and explicitly constructs a tie outcome to validate `winners()` in a multi-winner scenario.

5.5. Functions test logic

Within each phase block of the matrix, the test code contains one function-specific `it(...)` case per public entry point. Each case is written to exercise the contract in the same order as the on-chain checks, and to validate three kinds of observable behavior: revert reasons for invalid calls, event emission for valid calls, and state consequences (directly or via invariants).

For functions that are phase-gated and/or chairperson-restricted, the generic "Wrong phase" / "Only chairperson" outcomes (including guard ordering) are already asserted by the phase-function matrix loop in Section 5.4. The function paragraphs below therefore focus on the *function-specific* edge cases and the success-path observables.

`register()`

In the Registration phase, the test performs a successful registration and checks `VoterRegistered`, then attempts a duplicate registration and expects "Already registered", and finally passes the zero address and expects "Zero address". A non-chair registration attempt is also included to confirm `onlyChair` enforcement (the phase/ordering aspect is covered by the matrix loop).

`setDeadlines()`

In the Registration phase, the test exercises three negative cases derived from the contract's deadline constraints: invalid ordering ("Invalid deadline order"), commit deadline in the past ("Commit deadline in past"), and reveal deadline in the past ("Reveal deadline in past"). It then executes one valid call and checks that `DeadlinesSet` is emitted. A non-chair call is included to confirm `onlyChair` enforcement (phase/ordering outcomes are covered by the matrix).

`addProposal()` **and** `renameProposal()`

Proposal management is allowed only during setup (Initialization or Registration) and is frozen once voting begins. The matrix loop already asserts phase gating for both functions; the dedicated tests focus on content and index constraints plus event emission.

For `addProposal()`, the test passes an empty string to validate "Empty name", then adds a non-empty proposal and checks that `ProposalAdded` is emitted.

For `renameProposal()`, the test validates an out-of-range index ("Bad index") and an empty name ("Empty name"), then renames proposal index 0 and checks `ProposalRenamed`. A non-chair call is included to confirm `onlyChair` enforcement.

`commitVote()`. The `commitVote()` test mirrors the contract's check order: `inState(Commit)` → commit window → registration status → non-zero commitment → "not already committed".

To make the test robust to encoding details, it uses the contract's own `computeCommitment` helper to produce the commitment hash for both a registered voter and an unregistered voter:

```
const cRv = await voting.connect(rv).computeCommitment(On, SALT_OK);
const cUv = await voting.connect(uv).computeCommitment(On, SALT_OK);
```

In the Commit phase, the test first commits as an unregistered account and expects "Not registered", then commits with `ZeroHash` and expects "Bad commitment". It then executes one successful commit from `rv` and checks `Committed`, followed by a second commit from the same account to confirm "Already committed".

Finally, the test validates the time-window guard using a single, repeatable pattern also used for `revealVote()`: read the on-chain deadline, mine a block at `deadline + 1`, then assert the expected expiry revert ("Commit phase ended" here).

`revealVote()`

It mirrors the contract's reveal check order: `inState(Reveal)` → reveal window → registration status → non-zero commitment → index bounds → hash match → "not already revealed".

In the Reveal phase (constructed by `buildPhase`), the registered voter already holds a valid commitment for proposal index 0 using `SALT_OK`. The test executes the following sequence: (1) unregistered reveal attempt → "Not registered"; (2) out-of-range index → "Bad index"; (3) wrong salt → "Commit mismatch"; (4) valid reveal → `emit Revealed`; (5) second reveal attempt → "Already revealed".

The reveal time window is validated with the same deadline-mining pattern as in `commitVote()` (reading `revealDeadline` and mining at `revealDeadline + 1`), expecting "Reveal phase ended".

One additional negative case cannot be reached using `buildPhase(Reveal)` because that helper commits once in order to enter Reveal. To test the revert "No commitment", the code creates a separate contract instance, registers a voter, sets deadlines, advances into Commit, lets the commit deadline expire, advances to Reveal without any successful commits, and then calls `revealVote` to confirm that the contract rejects reveals without a stored commitment.

`advanceIfExpired()`

The `advanceIfExpired()` test validates both its phase restriction and its two-way behavior within Commit/Reveal. For phases other than Commit and Reveal, the function is expected to revert with "No auto-advance". Within Commit and Reveal, the test first

calls `advanceIfExpired()` immediately and expects "Not expired" to confirm that it does not advance prematurely.

It then forces expiration via timestamp mining. In Commit, mining to `commitDeadline + 1` and calling `advanceIfExpired()` is expected to emit `AutoAdvanced` and set the state to `Reveal`. In `Reveal`, mining to `revealDeadline + 1` is expected to emit `AutoAdvanced` and set the state to `Finalized`. In both cases, invariants are checked after the transition.

`results()` **and** `winners()`

The matrix loop asserts strict phase gating for both calls; the dedicated tests focus on output structure and a tie scenario.

In the `Finalized` phase built by `buildPhase(Finalized)`, the suite asserts that `results()` returns the proposal name array and the vote count array in matching order, and that `winners()` returns the expected index list. Because the phase builder creates a single reveal for proposal 0, the expected output is a single winner `[0]` with vote counts `[1,0]`.

The separate end-to-end flow test constructs a tie by registering two voters, having them commit to different proposals, revealing both, finalizing, and then asserting that `winners()` returns both indices `[0,1]`. This case is included because it validates multi-winner output behavior that cannot be observed in the minimal one-vote baseline used by `buildPhase(Finalized)`.

5.6. Global invariant checks

In addition to functions assertions, the test suite enforces a small set of global invariants after individual test actions. The motivation is that function-level checks (e.g., a specific revert reason or event emission) may still miss unintended state corruption that only becomes visible when multiple storage variables are considered together. Therefore, the matrix suite calls `assertInvariants(voting)` at the end of each test case (including branches that deliberately revert), turning the invariants into a continuous regression guard across the entire API surface.

The invariants are derived directly from the contract's counter-based workflow and storage layout:

- **Counter ordering.** The reveal count must never exceed the commit count, and the commit count must never exceed the registration count: `revealedCount ≤ committedCount ≤ registeredCount`. This captures the intended ordering "register → commit → reveal".
- **Registered address list consistency.** The length of the on-chain voter address list must match the registration counter: `voterAddresses().length == registeredCount`. This prevents silent divergence between a counter and the underlying address registry.

- **Tally conservation.** The sum of per-proposal `voteCount` values must equal `revealedCount`. Each successful reveal is expected to increment exactly one proposal's tally and the global revealed counter, and this invariant detects any mismatch.
- **Per-voter consistency.** For every address returned by `voterAddresses()`, the voter record returned by `getVoter` must satisfy: `registered` is true; if `committed` is true then the stored commitment hash is not the zero hash; and if `revealed` is true then `committed` is also true and the stored vote index is within `[0, proposals.length)`.

Because these checks are executed repeatedly across many distinct paths (valid calls, invalid calls, and time-dependent branches), they provide strong protection against subtle bugs that would otherwise require a much larger number of bespoke state assertions in each individual test case.

5.7. `changeState` failure coverage checklist

Besides the phase-function matrix (Sections 5.4 and 5.5), the test code contains a dedicated suite for `changeState`. The goal of this suite is not to repeat the general phase gating checks, but to ensure that each distinct transition guard implemented in `changeState` is reachable and covered by at least one explicit scenario. To achieve this, the suite maintains a set of observed revert messages and verifies in an after hook that the set equals a predefined list of expected messages.

The checklist covers the following revert reasons and the corresponding triggering scenarios:

- Only chairperson: a non-chair account attempts any state change (e.g., Initialization → Registration).
- Must move by one: the chairperson attempts to skip a phase (e.g., Initialization → Commit).
- No proposals: the contract is deployed with an empty proposal list and the chairperson attempts Registration → Commit.
- Deadlines not set: proposals exist, the contract is in Registration, but deadlines have not been configured; the chairperson attempts Registration → Commit.
- Commit deadline in past: deadlines are set with a commit deadline close to the current timestamp; the test advances the local chain time past that commit deadline and then attempts Registration → Commit. This confirms that the transition is rejected when the commit window is already expired at the time of entering Commit.
- Commit still open: the contract is advanced into Commit with valid deadlines, but time has not passed beyond the commit deadline; an attempt to advance Commit → Reveal is rejected.
- Reveal still open: the contract is advanced into Reveal (with at least one valid

```
Running Mocha tests

Voting - matrix + invariants
✓ sanity: deployment getters (122ms)
✓ invariant-preserving full flow (commit-reveal-finalize) with tie (233ms)
phase * role/identity matrix
phase = Initialization
✓ register(): phase gating + zero address + duplicate (63ms)
✓ setDeadlines(): phase gating + invalid order + past + success (54ms)
✓ addProposal(): phase gating + empty name + success (61ms)
✓ renameProposal(): phase gating + bad index + empty name + success (59ms)
✓ commitVote(): phase gating + not registered + bad commitment + double commit + time window (54ms)
✓ revealVote(): phase gating + not registered + bad index + mismatch + double reveal + time window + no commitment (50ms)
✓ advanceIfExpired(): phase gating + expired path (47ms)
✓ results() / winners(): phase gating + finalized outputs (53ms)
phase = Registration
✓ register(): phase gating + zero address + duplicate (76ms)
✓ setDeadlines(): phase gating + invalid order + past + success (67ms)
✓ addProposal(): phase gating + empty name + success (61ms)
✓ renameProposal(): phase gating + bad index + empty name + success (66ms)
✓ commitVote(): phase gating + not registered + bad commitment + double commit + time window (55ms)
✓ revealVote(): phase gating + not registered + bad index + mismatch + double reveal + time window + no commitment (52ms)
✓ advanceIfExpired(): phase gating + expired path (49ms)
✓ results() / winners(): phase gating + finalized outputs (50ms)
phase = Commit
✓ register(): phase gating + zero address + duplicate (78ms)
✓ setDeadlines(): phase gating + invalid order + past + success (81ms)
✓ addProposal(): phase gating + empty name + success (75ms)
✓ renameProposal(): phase gating + bad index + empty name + success (81ms)
✓ commitVote(): phase gating + not registered + bad commitment + double commit + time window (108ms)
✓ revealVote(): phase gating + not registered + bad index + mismatch + double reveal + time window + no commitment (76ms)
✓ advanceIfExpired(): phase gating + expired path (89ms)
✓ results() / winners(): phase gating + finalized outputs (73ms)
phase = Reveal
✓ register(): phase gating + zero address + duplicate (104ms)
✓ setDeadlines(): phase gating + invalid order + past + success (102ms)
✓ addProposal(): phase gating + empty name + success (115ms)
✓ renameProposal(): phase gating + bad index + empty name + success (97ms)
✓ commitVote(): phase gating + not registered + bad commitment + double commit + time window (93ms)
✓ revealVote(): phase gating + not registered + bad index + mismatch + double reveal + time window + no commitment (184ms)
✓ advanceIfExpired(): phase gating + expired path (92ms)
✓ results() / winners(): phase gating + finalized outputs (104ms)
phase = Finalized
✓ register(): phase gating + zero address + duplicate (124ms)
✓ setDeadlines(): phase gating + invalid order + past + success (116ms)
✓ addProposal(): phase gating + empty name + success (108ms)
✓ renameProposal(): phase gating + bad index + empty name + success (127ms)
✓ commitVote(): phase gating + not registered + bad commitment + double commit + time window (130ms)
✓ revealVote(): phase gating + not registered + bad index + mismatch + double reveal + time window + no commitment (118ms)
✓ advanceIfExpired(): phase gating + expired path (111ms)
✓ results() / winners(): phase gating + finalized outputs (122ms)

Voting - changeState failures + reachable coverage checklist
✓ only chairperson (40ms)
✓ must move by one
✓ no proposals (47ms)
✓ deadlines not set (45ms)
✓ commit deadline in past (changeState to commit after deadline passes) (55ms)
✓ commit still open (commit -> reveal blocked) (70ms)
✓ reveal still open (reveal -> finalized blocked) (183ms)

49 passing (5s)
```

Figure 5.1. Recorded output of running `npm hardhat test` for the Voting contract test suite.

commit so that Reveal is reachable), but time has not passed beyond the reveal deadline; an attempt to advance Reveal → Finalized is rejected.

This suite is intentionally narrow: it treats the set of transition guards as a stability contract and prevents accidental weakening of `changeState` semantics by requiring that every expected guard remains test-covered.

5.8. Test execution results

The test suite is executed using Hardhat’s test runner via `npm hardhat test` (Section 5.2). The recorded output of a full run is included in Figure 5.1.

6. Conclusions

This thesis explored how to design and implement a practical decentralized voting system for small and medium-sized elections on an Ethereum blockchain. The objective was to reduce the amount of trust placed in a centralized server by making the election workflow and outcome verifiable from a public blockchain, while still keeping the system deployable with standard tools such as a web browser and a wallet.

The core contribution is a concrete protocol and contract design based on an explicit phase state machine and a commit–reveal workflow. Each deployment of the Voting contract represents one election instance and progresses through Initialization, Registration, Commit, Reveal, and Finalized phases. The contract acts as the sole authority for election rules: it restricts administrative actions to the chairperson, enforces phase and time window constraints via deadlines, records voter commitments, and counts votes only when a reveal matches a prior commitment. The commitment construction includes the voter address, contract address and chain id, which prevents simple replay across voters, contracts, or networks.

A second contribution is an end-to-end decentralized application that treats the contract as the source of truth. The front-end renders the current on-chain state (phase, proposals, deadlines, participation counters, and final results) and prepares transactions, while MetaMask handles key management and signing. To support commit–reveal, the client generates a random salt locally and persists it for later reveal, but it does not store any authoritative election data.

The implementation was evaluated primarily through systematic smart contract testing on a local Hardhat network. The main suite executes a phase–function matrix that attempts every public entry point in every phase, asserting correct revert reasons, event emission, and role/phase guard ordering. In addition, invariant checks are executed after each test step to detect unintended state corruption across interactions. Deadline-sensitive behavior is made deterministic by controlling block timestamps, allowing tests to cover boundary conditions around commit and reveal expiry. Together, these tests provide evidence that the state machine, access control, and commit–reveal verification enforce the intended workflow and that the final tally can be derived from stable on-chain state in the Finalized phase.

At the same time, the system has limitations that follow from its scope and the constraints of public blockchains. Commit–reveal delays disclosure of choices but does not provide strong voter anonymity: addresses and participation remain public, and missing reveals lead to uncounted ballots. Deadlines rely on `block.timestamp`, which is not precise timing. The design also assumes moderate election sizes due to linear scans in some readouts and due to the cost of on-chain interactions. The chairperson retains significant administrative power during setup (voter enrollment, proposal management, and deadline configuration), and each election requires a fresh contract deployment.

Future work could improve robustness and applicability by:

- introducing penalties to reduce missing reveals (e.g., deposits);
- adding stronger privacy mechanisms beyond commit–reveal;
- providing a factory or registry contract to manage multiple elections and standardize deployments;
- extending evaluation with gas/performance measurements and a security review;
- adding deterministic end-to-end front-end tests to complement the contract test suite.

References

- [1] S. Nakamoto, *Bitcoin: A peer-to-peer electronic cash system*, <https://bitcoin.org/bitcoin.pdf>, 2008.
- [2] M. Iqbal and R. Matulevičius, “Exploring sybil and double-spending risks in blockchain systems”, *IEEE Access*, vol. 9, pp. 76 153–76 177, 2021. DOI: 10.1109/ACCESS.2021.3081998.
- [3] V. Buterin, *A next-generation smart contract and decentralized application platform*, <https://ethereum.org/en/whitepaper/>, 2014.
- [4] G. Wood, “Ethereum: A secure decentralised generalised transaction ledger”, Ethereum Foundation, Tech. Rep., 2014. [Online]. Available: <https://ethereum.github.io/yellowpaper/paper.pdf>.
- [5] Ethereum Foundation, *Ethereum developer documentation*, <https://ethereum.org/developers/docs/>.
- [6] Solidity Team, *Solidity documentation*, <https://docs.soliditylang.org/>.
- [7] Nomic Foundation, *Hardhat ignition documentation*, <https://hardhat.org/ignition/docs/>.
- [8] The Ethers Project, *Ethers.js documentation*, <https://docs.ethers.org/>.
- [9] wevm, *Wagmi documentation*, <https://wagmi.sh/react/getting-started>.
- [10] wevm, *Viem documentation*, <https://viem.sh/docs/getting-started>.
- [11] Vercel, *Next.js documentation*, <https://nextjs.org/docs>.
- [12] MetaMask, *Metamask developer documentation*, <https://docs.metamask.io/>.
- [13] Nomic Foundation, *Hardhat 3 documentation*, <https://hardhat.org/docs/>.

Source Code Repository

<https://github.com/Fantasy7780/voting-system>

List of Figures

2.1	Bitcoin block structure[2].	10
2.2	Example of UTXO.	11
2.3	A transaction causes a state change on Ethereum [5].	12
2.4	Ethereum Virtual Machine [5].	14
3.1	High-level architecture and interaction paths.	17
3.2	Phase state machine of a single election contract instance.	20
3.3	Front-end state model.	28
5.1	Recorded output of running <code>npm hardhat test</code> for the Voting contract test suite.	48

List of Tables

3.1	Summary of on-chain storage elements and their roles.	23
3.2	Public contract API grouped by function category.	24
3.3	Contract events and their off-chain purpose.	25